



# NPTEL Course on “Computer System Optimization for ML”

Lecture #13



# Agenda

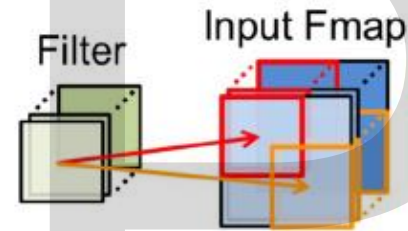
- **Recap**
- **DNN accelerators**

NPTEL

# Types of Data Reuse in DNN

## Convolutional Reuse

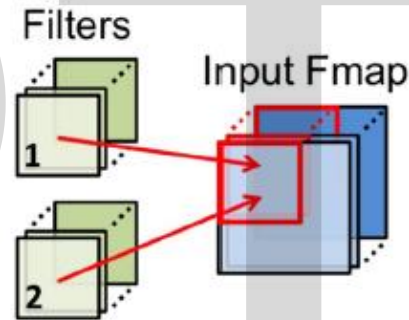
CONV layers only  
(sliding window)



Reuse: **Activations**  
**Filter weights**

## Fmap Reuse

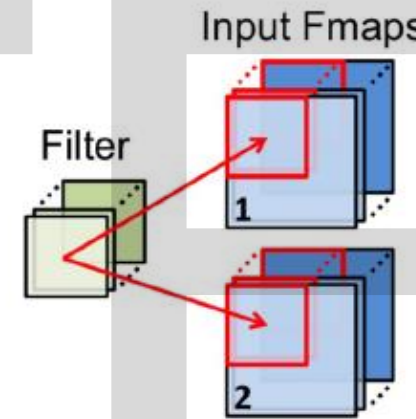
CONV and FC layers



Reuse: **Activations**

## Filter Reuse

CONV and FC layers  
(batch size > 1)



Reuse: **Filter weights**

- If all types of data reuse is utilized, DRAM access in AlexNet can be reduced from **2896M** to **61M** (best and hypothetical case)

# Cost of different data flow

|               | OS          | WS          | IS          | Min         |
|---------------|-------------|-------------|-------------|-------------|
| MACs          | $E \cdot R$ | $E \cdot R$ | $E \cdot R$ | $E \cdot R$ |
| Weight Reads  | $E \cdot R$ | $R$         | $E \cdot R$ | $R$         |
| Input Reads   | $E \cdot R$ | $E \cdot R$ | $E$         | $E$         |
| Output Reads  | 0           | $E \cdot R$ | $E \cdot R$ | 0           |
| Output Writes | $E$         | $E \cdot R$ | $E \cdot R$ | $E$         |

Assume:  $W \sim E$

# Agenda

- Recap
- **DNN accelerators**
  - DianNao [1]
  - DaDianNao [2]
  - ShiDianNao [3]
  - Eyeriss [4]

[1] Chen, Tianshi, et al. "Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning." ACM SIGARCH Computer Architecture News 42.1 (2014): 269-284.

[2] Chen, Yunji, et al. "Dadiannao: A machine-learning supercomputer." *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 2014.

[3] Du, Zidong, et al. "ShiDianNao: Shifting vision processing closer to the sensor." *Proceedings of the 42nd Annual International Symposium on Computer Architecture*. 2015.

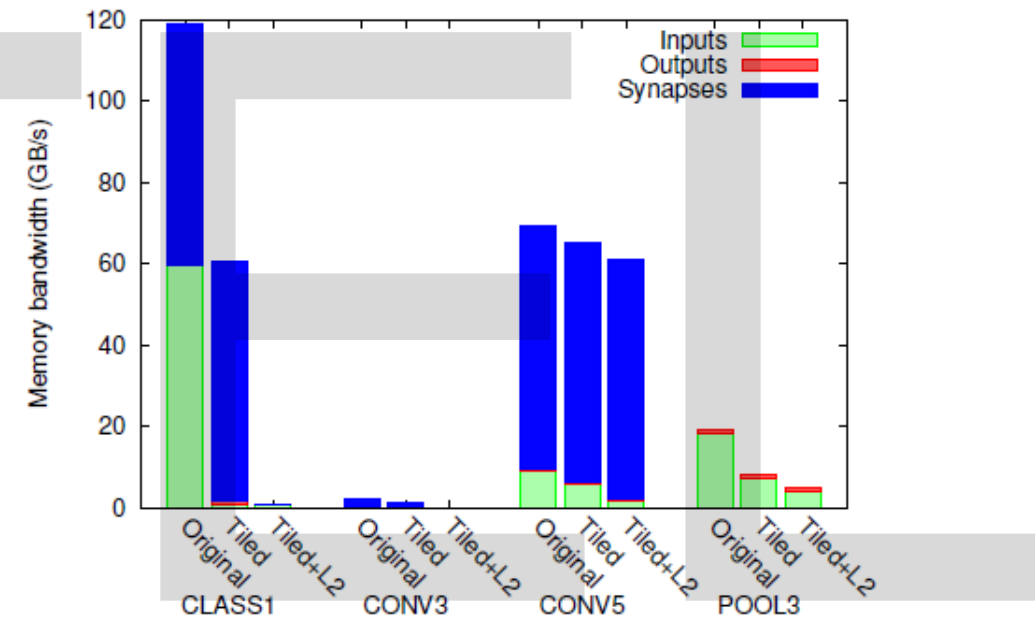
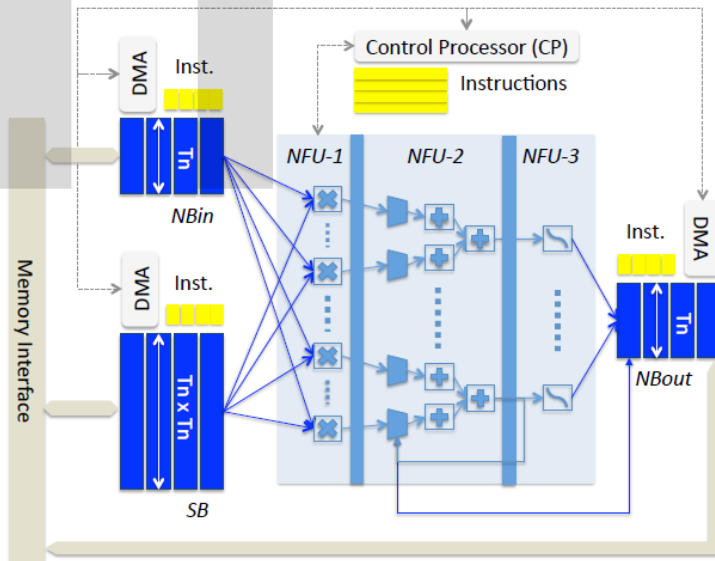
[4] Chen, Yu-Hsin, et al. "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks." *IEEE journal of solid-state circuits* 52.1 (2016): 127-138.

# Accelerator: Why/What?

- **Drawbacks of CPUs**
  - Less parallelism
- **Drawback of GPU**
  - Less memory
- **Accelerators**
  - Application specific
  - Tailored only for DNNs
  - **Mostly inference**
- **Major performance metrics**
  - Inference latency, energy, area

# DianNao (Computer) [1]

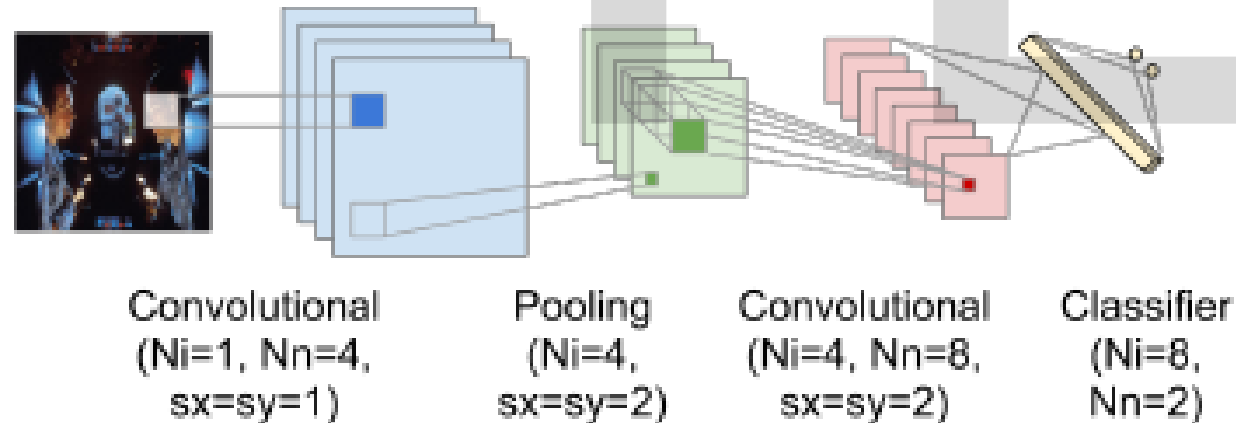
- One of the earliest accelerator for DNN models
- Tiling to reduce memory usage
  - Classifier layer
  - Convolutional layer
  - Pooling layer
- An architecture tailored for DNN inference



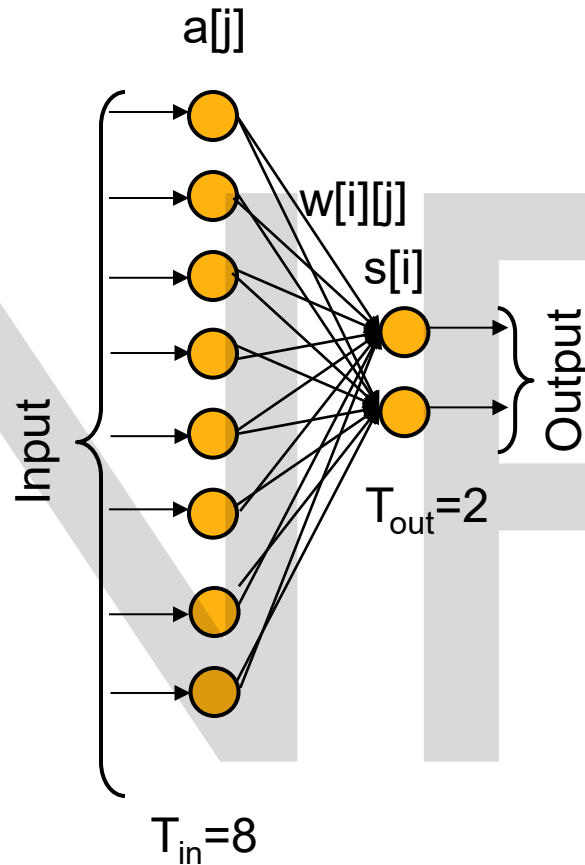
[1] Chen, Tianshi, et al. "Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning." ACM SIGARCH Computer Architecture News 42.1 (2014): 269-284.

# General Structure of a DNN

- Sequential layers
- Feature maps
- Convolutional (nonlinear after the output), pooling, classifier
  - Same kernels sometimes/private kernels
  - Pooling: average and max
  - Classifier is a multi/single layer perceptron



# Classifier Layer



## ■ Notations

- $T_{in}$  = Number of input neurons
- $a[j]$  = Activation (value) at  $j^{th}$  input neuron
- $T_{out}$  = Number of output neurons
- $w[i][j]$  = Weight from  $j^{th}$  input neuron to  $i^{th}$  output neuron
- $s[i]$  = Sum at  $i^{th}$  output neuron

## ■ $T_{in}=8, T_{out}=2$ in this example

## ■ Implementation

## ■ #memory accesses?

```
for (i=0; i<Tout; i++) {  
    for (j=0; j<Tin; j++) {  
        s[i] += a[j].w[i][j];  
    }  
}
```

# Memory accesses in classifier layer

## ■ Notations

- $T_{in}$  = Number of input neurons
- $a[j]$  = Activation (value) at  $j^{th}$  input neuron
- $T_{out}$  = Number of output neurons
- $w[i][j]$  = Weight from  $j^{th}$  input neuron to  $i^{th}$  output neuron
- $s[i]$  = Sum at  $i^{th}$  output neuron

## ■ #memory accesses?

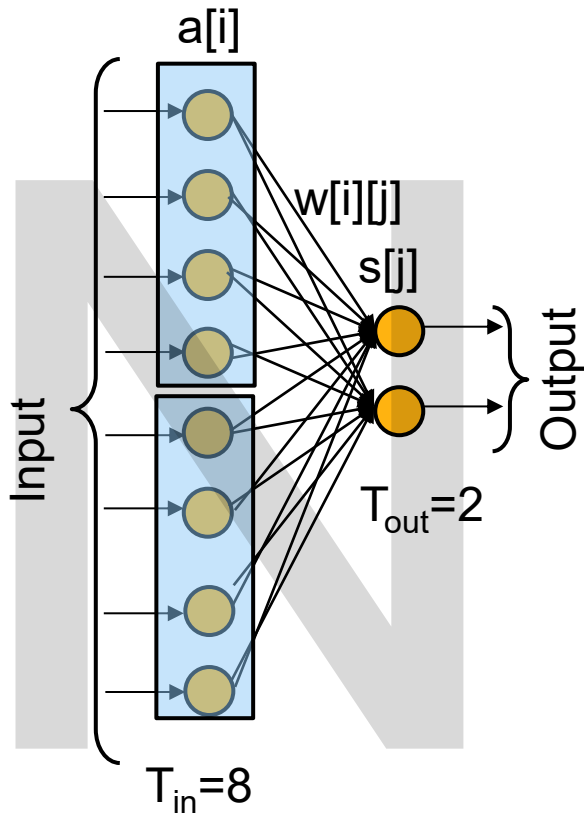
- Activations:  $T_{in} \times T_{out}$
- Weights:  $T_{in} \times T_{out}$
- Sum:  $T_{out}$
- **Total:  $T_{in} \times T_{out} + T_{in} \times T_{out} + T_{out}$**

## ■ Drawback

- Number of input neurons ( $T_{in}$ ) can be very large (10 – 1L)
- Can not fit in L1

```
for (i=0; i<Tout; i++) {  
    for (j=0; j<Tin; j++) {  
        s[i] += a[j].w[i][j];  
    }  
}
```

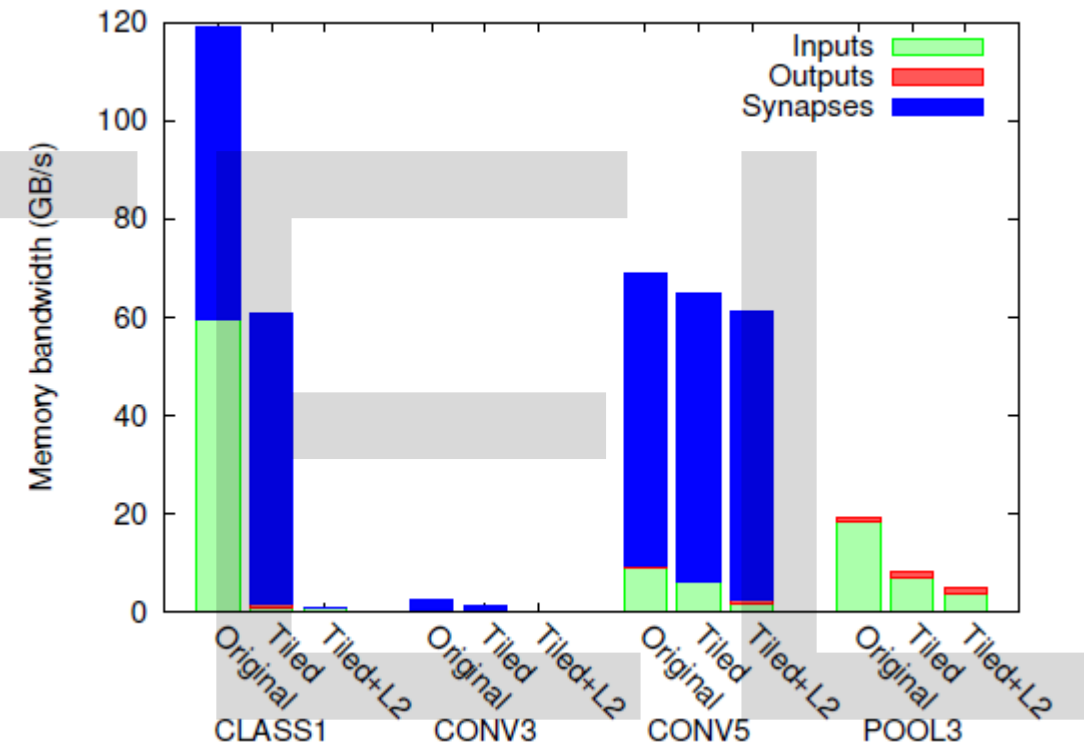
# How to Reduce the Memory Access



- **Rather, how to use L1?**
- **Tiling**
  - Divide the input neurons into tiles
  - Store the tiles in L1
  - Compute the contribution from each tile separately
  - Accumulate the outputs
- **Trade off**
  - Number of times  $a[i]$  needs to be fetched from main memory is less (+)
  - Number of times  $s[j]$  needs to be fetched is more (-)

# Tiling for Convolutional and Pooling Layers

- **Tiling for convolutional layers**
  - Input feature maps
  - Output feature maps
- **Pooling layers**
  - No kernels
  - Tiling does not improve the BW requirement drastically





# THANK YOU

Slide Courtesy: Murali Annavaram (University of Southern California), R. Govindarajan (IISc), Arkaprava Basu (IISc), Jayashree Mohan (Microsoft Research)

